1. Data

Feature types. Nominal ($=, \neq$) ID, eye color, zip code. Ordinal ($<, >$) rankings, grades A–D, so coding 1–4 is fine. Interval ($+, -$) Celsius, Fahrenheit, credit score 300–850, no true zero. Ratio ($\times, \div$) length, counts, Kelvin, has a true zero. Nominal and ordinal are categorical, interval and ratio are numeric. Feature = variable, field, dimension, attribute. Sample = record, point, case, object, entity, instance.

Data types. Record (document = term *counts*, transaction = 0/1 basket). Graph (WWW, social nets, molecules). Ordered (spatial, temporal, sequential, genetic sequence).

Messy data. Salary -100 is noise. Missing values, ignoring the record is not effective, the same-class mean beats the overall mean. Duplicates, exact or soft match, classifier, or clustering. Cleaning is about 60% of the time.

Reduction. Same or almost the same results with less data. Sampling with replacement can pick an object twice. Curse of dimensionality, more features means sparser data (bins^d cells) and more uniform, less meaningful distances, ratio $(\max - \min)/\min$ drops. Redundant feature repeats another (price and tax), irrelevant one is useless (student ID for GPA).

One-hot. k categories give k columns with one 1. Use for nominal, since integers fake an order.

Normalize. z-score (standardization) $z = \frac{x-\mu}{\sigma}$, $\sigma = \sqrt{\frac{1}{n-1}\sum(x_i-\mu)^2}$, range $(-\infty, \infty)$. GPA 4, 3.5, 3 gives 1, 0, -1. Min-max $z = \frac{x-\min}{\max-\min} \in [0,1]$, sensitive to outliers.

2. KNN and evaluation

$\mathcal{D}^{train} \cap \mathcal{D}^{dev} \cap \mathcal{D}^{test} = \emptyset$. 1-NN copies the label of the nearest sample, boundary very sensitive to noise. KNN votes $v_c = \sum_{k \in knn(\mathbf{x})} \mathbb{I}(y_k = c)$, $\hat{y} = \arg\max_c v_c$. **K too small** sensitive to noise, overfits. **K too large** neighborhood includes other classes, underfits. **Cost** one distance $O(D)$, all $O(ND)$, find K closest $O(NK)$, total $O(ND + NK)$. Keeps all training data in memory, sensitive to noisy features, bad in high dimensions. **Non-parametric**, no model parameters. K and the metric are hyperparameters, set by people, not learned. Linear regression weights are model parameters.

Weighted KNN. $w_k = \frac{e^{-d_k}}{\sum_{j \in knn} e^{-d_j}}$, $v_c = \sum w_k \mathbb{I}(y_k = c)$. Closer means bigger weight, weights sum to 1.

Validation. Tune on dev, test only once at the end. Early stopping uses validation, never test. **CV.** Split *train* into S folds, $E = \frac{1}{S}\sum E_s$, pick best average, retrain on all train, then test.

Distances. Euclid $\sqrt{\sum(x_k - y_k)^2}$, normalize if scales differ. Minkowski $(\sum |x_k - y_k|^r)^{1/r}$, $r = 1$ Manhattan (L1), $r = 2$ Euclid, $r \to \infty$ max difference. Cosine $\frac{A \cdot B}{\|A\|\|B\|} \in [-1, 1]$, ignores length.

Confusion. rows truth, columns prediction.

	Pred +	Pred −
True +	TP	FN
True −	FP	TN

Acc $\frac{TP+TN}{ALL}$, Err = 1−Acc.

Prec $\frac{TP}{TP+FP}$ (of predicted +), Rec $\frac{TP}{TP+FN}$ (of real +). $F_1 = \frac{2PR}{P+R}$ harmonic. $F_\beta = \frac{(1+\beta^2)PR}{\beta^2 P+R}$, $\beta \to 0$ gives P, $\beta \to \infty$ gives R. Specificity $\frac{TN}{TN+FP}$, FPR $\frac{FP}{FP+TN}$. **Precision** when FP costly (spam). **Recall** when FN costly (Covid, cancer). Imbalanced 99% acc by predicting all negative is useless.

Multi-class. For class c, TP is the diagonal, FN the rest of row c, FP the rest of column c, TN = total −TP−FN−FP. Prec_c = diag / column sum. Rec_c = diag / row sum. **Macro** average the per-class scores, every class equal, use for imbalanced data. **Micro** add the per-class tables then compute, dominated by frequent classes. Micro precision = micro recall = accuracy.

3. PCA

$\mathbf{y} = U^\top \mathbf{x}$, $U \in \mathbb{R}^{D \times M}$, $U^\top U = I$ (not $UU^\top$). Goals max variance of projected data AND min reconstruction error, achieves both. Unsupervised. Reduction keeps all features mixed, selection keeps a subset. **Zhu layout** $X \in \mathbb{R}^{p \times n}$, rows features, columns samples.

1. $\bar{X} = \frac{1}{n} X\mathbf{1}$, $\tilde{X} = X - \bar{X}\mathbf{1}^\top$ (each feature minus its own mean)
2. SVD $\tilde{X} = U\tilde{\Sigma}V^\top$, $\sigma_1 \geq \cdots \geq 0$
3. $G = U_k \in \mathbb{R}^{p \times k}$, first k columns of U
4. $Z = G^\top \tilde{X} \in \mathbb{R}^{k \times n}$, row k of Z is the k-th PC

Rows or columns? Read the question. "Rows represent samples" or $X \in \mathbb{R}^{N \times D}$ means flip.

	Rows features (X is $p \times n$, Zhu)	**Rows samples** (X is $N \times D$)
Center	each row's mean	each column's mean
SVD of	$\tilde{X}$	$\tilde{X}^\top$
U	$p \times p$	$D \times D$
$G = U_k$	$p \times k$	$D \times k$
Z	$G^\top \tilde{X}$, $k \times n$	$G^\top \tilde{X}^\top$, $k \times N$

Check, G has one row per *feature* and Z one entry per *sample*. A U the size of the sample count is the wrong one. Cov $C = \frac{1}{n}\tilde{X}\tilde{X}^\top$, eigenvalues of $\tilde{X}\tilde{X}^\top$ are σ_i^2. Signs can flip. Max variance works with X or $\tilde{X}$, min reconstruction only with $\tilde{X}$. Reconstruction $GG^\top\tilde{X}$, error $\sqrt{\sum_{i>k}\sigma_i^2}$. $G^\top G = I_k$, $GG^\top \neq I$. PCA is a linear autoencoder with shared weights, no bias. **Choose** k by a downstream task, where σ gets small, or smallest k with $\frac{\sum_{i \leq k}\sigma_i}{\sum \sigma_i} \geq 95\%$ or 99% (plain σ).

3. Decision trees

Internal node tests a feature, branch is an outcome, leaf is a class, root to leaf is a rule. Many trees can fit. Learning is induction, applying is deduction. Hunt, CART, ID3, C4.5, SLIQ, SPRINT. Greedy. $Entropy(t) = -\sum_j p(j|t)\log_2 p(j|t)$, 0 pure, max $\log_2 n_c$ (1 for 2 classes) when even. Larger entropy means *less* homogeneous. $GAIN = Ent(p) - \sum_i \frac{n_i}{n} Ent(i)$, pick max. Splits nominal multi-way or subsets, ordinal must keep order, continuous threshold or ranges. Stop when pure, early stop for overfitting. Cheap, fast, interpretable.

4. Linear regression

$f(\mathbf{x}) = w_0 + \sum w_d x_d = \mathbf{w}^\top \mathbf{x}$, $\mathbf{x}, \mathbf{w} \in \mathbb{R}^{D+1}$ (leading 1), $y \in \mathbb{R}$. $RSS(\mathbf{w}) = \sum_n (y_n - \mathbf{w}^\top \mathbf{x}_n)^2 = (\mathbf{y} - X\mathbf{w})^\top(\mathbf{y} - X\mathbf{w})$. $\mathbf{w}^* = \arg\min_{\mathbf{w}} RSS(\mathbf{w})$ (plain min is a number, so F). $\nabla RSS = -2X^\top\mathbf{y} + 2X^\top X\mathbf{w} = 0 \Rightarrow \mathbf{w}^{OLS} = (X^\top X)^{-1}X^\top\mathbf{y}$. Convex, Hessian $2X^\top X$ PSD, so local min is global. Inverse costs $O((D+1)^3)$, grows with D not N, so use GD (approximate). **GD.** $\mathbf{w} := \mathbf{w} - \alpha\nabla J$. α too small slow, too large overshoots and can diverge. ϵ small means more iterations, better solution. Non-convex result depends on start. α is about convergence, not overfitting. Batch $\mathbf{w} := \mathbf{w} - \alpha(X^\top X\mathbf{w} - X^\top\mathbf{y})$ all data. SGD one sample $\mathbf{g}_i = (\mathbf{x}_i^\top\mathbf{w} - y_i)\mathbf{x}_i$. Mini-batch a subset, good compromise. Epoch is the outer pass, iteration one update. **Basis.** $\phi\colon \mathbb{R}^D \to \mathbb{R}^M$, $\mathbf{w}^{OLS} = (\Phi^\top\Phi)^{-1}\Phi^\top\mathbf{y}$, still linear in $\mathbf{w}$. Poly $M = 0, 1$ underfit, 3 good, 9 overfits. **Overfit.** low train error, high test error. Fix with more data, regularization, early stopping. Train error always falls with complexity or steps, test falls then rises. **Regularization.** $\lambda \geq 0$ penalizes big weights, since big w_i let the model swing and fit noise. L2 $J = RSS + \lambda\sum_d w_d^2 = RSS + \lambda\|\mathbf{w}\|_2^2$. L1 $J = RSS + \lambda\sum_d |w_d| = RSS + \lambda\|\mathbf{w}\|_1$. L2 gradient $\nabla J = -2X^\top\mathbf{y} + 2X^\top X\mathbf{w} + 2\lambda\mathbf{w} = 0$ gives the closed form $\mathbf{w}^* = (X^\top X + \lambda I)^{-1}X^\top\mathbf{y}$, and with a basis $\mathbf{w}^* = (\Phi^\top\Phi + \lambda I)^{-1}\Phi^\top\mathbf{y}$. Both are OLS at $\lambda = 0$. For GD use $\mathbf{w} := \mathbf{w} - \alpha(X^\top X\mathbf{w} - X^\top\mathbf{y} + \lambda\mathbf{w})$. $M = 9$ example, $\lambda = 0$ overfits, $\lambda = e^{-18}$ fits, $\lambda = 1$ underfits. w_1 goes 232.37 → 4.74 → −0.05. Bigger λ simpler model and higher train error. Tune λ on validation. Logistic L2 and NN weight decay are listed in their sections.

5. Logistic regression

Bernoulli $p(y|\theta) = \theta^y(1-\theta)^{1-y}$, likelihood is a product, log-likelihood a sum, MLE *maximizes*, $\theta^* = \frac{\sum y_i}{n}$. $p(y=1|\mathbf{x}) = \sigma(\mathbf{w}^\top\mathbf{x})$, $\sigma(\eta) = \frac{1}{1+e^{-\eta}} \in (0,1)$, $\sigma' = \sigma(1-\sigma)$. Predict 1 if > 0.5 (threshold can change), boundary $\mathbf{w}^\top\mathbf{x} = 0$. Cross-entropy (negative log-likelihood) $\mathcal{E} = -\sum[y_n \log\sigma_n + (1-y_n)\log(1-\sigma_n)]$, convex, no closed form, use GD. $\nabla\mathcal{E} = \sum_n(\sigma(\mathbf{w}^\top\mathbf{x}_n) - y_n)\mathbf{x}_n$, $+2\lambda\mathbf{w}$ with L2. Basis works, replace $\mathbf{x}$ by $\phi(\mathbf{x})$. **Softmax.** one $\mathbf{w}_c$ per class, $p(y=c|\mathbf{x}) = \frac{e^{\mathbf{w}_c^\top\mathbf{x}}}{\sum_{c'} e^{\mathbf{w}_{c'}^\top\mathbf{x}}}$, sums to 1, $\hat{y} = \arg\max$.

Multi-class uses softmax, not sigmoid (0.8, 0.6, 0.3 sum 1.7). Loss is − log of the true class's probability only.

5. Neural networks

Perceptron $\hat{y} = s(\mathbf{w}^\top\mathbf{x})$, many choices of s, classification or regression. AND $s(x_1 + x_2 - 1.5)$. XOR not linearly separable, needs an MLP. MLP is feed-forward, non-linear, universal approximator. p.63 net $z_1 = s(-30 + 20x_1 + 20x_2)$ (AND), $z_2 = s(10 - 20x_1 - 20x_2)$ (NOR), out $s(-10 + 20z_1 + 20z_2)$ gives 1,0,0,1 so XNOR. Forward computes loss, backprop finds each node's responsibility (chain rule), GD updates. More layers and neurons mean more capacity, can overfit, so never "always better". Fix with fewer units, early stopping (small init weights), weight decay, a penalty on big weights ($\lambda = .001$ jagged boundary, .1 almost straight). **Activations** Linear adds no non-linearity. Sigmoid (0,1), saturates, not zero-centered. Tanh $= 2\sigma(2x) - 1 \in (-1, 1)$, saturates, zero-centered. ReLU $\max(0, x)$ non-saturating, about 6x faster, cheap, can die. Leaky ReLU ax for $x < 0$, not always consistent.

6. Deep learning

End-to-end, learns features from raw data, relies on big data, differentiable layers, no feature/classifier split. No strict depth rule. **Challenges** compute (distributed, compression, hardware), overfitting (dropout, weight decay, early stop), blackbox (interpretable ML), safety (safe AI), ethics (responsible AI). **CNN.** assumes images, filters are learned weights. $h[m,n] = \sum_{k,l} g[k,l]f[m+k, n+l]$, no flip. $N \times N$ with $F \times F$, no pad, gives $N - F + 1$. General $\frac{N-F+2P}{S} + 1$. Vertical edge $\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$, horizontal is its transpose, $\frac{1}{9}$ box blurs. Layers learn edges, parts, objects. Conv, subsample, conv, subsample, fully connected. **RNN.** $h_t = f_W(h_{t-1}, x_t)$, same W every step. Many-to-one (spam), many-to-many. Captioning is CNN then RNN. CNN and RNN can be combined. **Transformer.** LM head is linear + softmax (not conv). Prefill reads the prompt at once, decode is one token at a time. Softmax goes on key-query scores, not values. Text or images.

6. Fairness

Data is biased and models inherit or amplify it. 2 of 4 face platforms fail, speech errors 2x. SP (demographic parity) $\hat{y} \perp s$, $\Delta_{SP} = |P(\hat{y}=1|s=0) - P(\hat{y}=1|s=1)|$, no y. EO equal TPR, $\Delta_{EO} = |P(\hat{y}=1|y=1, s=0) - P(\hat{y}=1|y=1, s=1)|$. Equalized odds, same for $y = 0$ and $y = 1$, so it implies EO. Individual $D(f(u), f(v)) \leq d(u, v)$. Training $\mathcal{L} = \mathcal{L}_{cls} + \alpha\Delta_{SP}$, bigger α fairer, trades accuracy.

Worked examples

Tree root split. 8 days, 4 Play yes, 4 no. Outlook Sunny (3Y,1N), Rain (1Y,3N). Windy True (0Y,2N), False (4Y,2N). Which feature goes at the root?

1. Parent entropy, 4Y 4N is an even split, so $H = 1$.
2. Outlook. Each side is 3 to 1, $H = -\frac{3}{4}\log_2\frac{3}{4} - \frac{1}{4}\log_2\frac{1}{4} = .811$. Weighted $\frac{4}{8}(.811) + \frac{4}{8}(.811) = .811$. Gain $1 - .811 = .189$.
3. Windy. True side is pure, $H = 0$. False side 4 to 2, $H(\frac{2}{3}) = .918$. Weighted $\frac{2}{8}(0) + \frac{6}{8}(.918) = .689$. Gain $1 - .689 = .311$.
4. Pick the bigger gain, so **Windy** is the root.

Fairness from a table. Hiring, s=1 men, s=0 women, 10 each, 5 qualified (y=1) each. Men hired 4 of 5 qualified, 2 of 5 unqualified. Women hired 2 of 5 qualified, 1 of 5 unqualified. Find Δ_{SP}, Δ_{EO}, equalized odds.

1. SP uses everyone. Men $6/10 = .6$, women $3/10 = .3$, $\Delta_{SP} = .3$.
2. EO uses only qualified, the TPR. Men $4/5 = .8$, women $2/5 = .4$, $\Delta_{EO} = .4$.
3. Equalized odds also checks unqualified, the FPR. Men $2/5 = .4$, women $1/5 = .2$, gap .2. Fair only if both gaps are 0, so it fails.

Logistic GD step. One sample $\mathbf{x} = [1, 2]$ (1 is the bias), $y = 1$, $\mathbf{w} = [0, 0]$, $\alpha = .5$. Do one update.

1. $\mathbf{w}^\top\mathbf{x} = 0$, so $\sigma(0) = .5$.
2. Error $\sigma - y = .5 - 1 = -.5$. Gradient $(\sigma - y)\mathbf{x} = [-.5, -1]$.
3. $\mathbf{w} \leftarrow \mathbf{w} - \alpha\nabla = [0, 0] - .5[-.5, -1] = [.25, .5]$.
4. Check, $\mathbf{w}^\top\mathbf{x} = 1.25$, $\sigma = .777$, moved toward the true label 1.

KNN, normalize first. Train A (age 20, income 30000, yes), B (age 60, income 31000, no). Test (22, 30600). 1-NN?

1. Raw distances, A $\sqrt{2^2 + 600^2} = 600$, B $\sqrt{38^2 + 400^2} = 402$, so B. Income swamps age.
2. Min-max with train min and max. Age $(22 - 20)/40 = .05$, income $(30600 - 30000)/1000 = .6$. A is $(0, 0)$, B is $(1, 1)$.
3. A $\sqrt{.05^2 + .6^2} = .602$, B $\sqrt{.95^2 + .4^2} = 1.031$, so **A, yes**. Normalizing flipped the answer.

Softmax and loss. Scores for classes 1, 2, 3 are 2, 1, 0. Predict, and give the loss if the true class is 2.

1. $e^2 = 7.389$, $e^1 = 2.718$, $e^0 = 1$, sum 11.107.
2. Probabilities .665, .245, .090, they sum to 1. Predict class 1, the arg max.
3. Loss is $-\ln$ of the true class's probability only, $-\ln .245 = 1.41$.

Macro/micro recall and F1. Same 3-class matrix as side 2.

1. Recall_c = diagonal / *row* sum, $28/30 = .933$, $25/30 = .833$, $23/30 = .767$, macro .844.
2. $\text{F1}_c = 2PR/(P+R)$ with P .800, .862, .885 gives .862, .847, .821, macro .843.
3. Micro recall $= 76/(76 + 14) = .844$, same as micro precision and accuracy.

True/False, statement then why

1. KNN weight $w_g = e^{-d(\mathbf{x},g)}/\sum_k e^{-d(\mathbf{x},k)}$, closer neighbor gets a bigger weight. **T** *Exactly Zhu's formula. Smaller d means bigger e^{-d}, and dividing by the sum makes weights add to 1.*
2. Letter grades A, B, C, D can be coded as 1, 2, 3, 4. **T** *Grades are ordinal, the order is real, so numbers keep it. One-hot is only needed for nominal (eye color).*
3. $X \in \mathbb{R}^{N\times D}$ (rows samples). Center, SVD the transpose, get $U \in \mathbb{R}^{N\times N}$, $G = U_k \in \mathbb{R}^{N\times k}$. **F** *G maps features to k dims so it must be $D \times k$. SVD of $X^\top$ gives U that is $D \times D$.*
4. After each epoch evaluate on the *test* set and stop when it stops improving. **F** *Early stopping uses the validation set. Test is used once at the end, or it leaks.*
5. Decreasing the learning rate α (all else fixed) counters overfitting. **F** *α only changes how fast or whether GD converges. Overfitting is fixed by λ, more data, or the stopping rule.*
6. Linear regression has a closed form, and with L2 it still does. **T** $(X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$.
7. Multi-class logistic regression uses a sigmoid per class for the probabilities. **F** *Softmax. Separate sigmoids don't sum to 1 (.8+.6+.3=1.7).*
8. The 0.5 threshold in binary logistic regression is fixed. **F** *Slide says it can change as we need.*
9. Linear regression solves $\mathbf{w}^* = \arg\min_{\mathbf{w}} RSS(\mathbf{w})$. **T** *With plain min it is F, min is a number not a vector.*
10. KNN is a parametric non-linear model. **F** *Non-parametric, nothing is learned. K is a hyperparameter.*
11. Non-linear basis can fit complex data so it is always preferred. **F** *"Always", it can overfit (degree 9).*
12. Smaller k in KNN gives a higher chance of overfitting. **T** *Small k follows noise points, jagged boundary.*
13. Test set 90 A, 5 B, 5 C. Micro-averaged metrics are preferred. **F** *Micro is dominated by the big class A. Macro weights classes equally, so it shows the bad small classes.*
14. More layers in an MLP always gives better accuracy. **F** *More capacity can overfit.*
15. CNN for images, RNN for sequences, they can't be used together. **F** *Layers stack, e.g. CNN per frame then RNN, or captioning.*
16. GD finds the global min if convex, only a local min if non-convex. **T** *Convex means local is global. Non-convex depends on the start.*
17. FN costly (cancer called healthy), so we want high precision. **F** *FN is in recall = $TP/(TP+FN)$, so focus on recall.*
18. Larger entropy means a more homogeneous node. **F** *Backwards. Entropy 0 is pure.*
19. Distances from a test sample to all N training samples, dim D, cost $O(ND)$. **T** *N distances, each $O(D)$. Full KNN adds $O(NK)$.*
20. Credit score (300 to 850) is an interval attribute. **T** *Differences mean something, no true zero, so ratios don't.*

Also seen. $\mathbf{w}^* = \min_{\mathbf{w}} RSS$ is **F**. Stop when *validation* stops improving is **T**.

Problem 1. KNN with built features

The question

Classify a movie as sci-fi or animation. Labeled movies are Avatar (sci), Big Hero 6 (ani), Captain America (sci), Despicable Me (ani), Ender's Game (sci), Frozen (ani). Watch history. Alice saw BH6, CapAm, Desp. Bob BH6, Desp, Frozen. Chris BH6, Desp, Ender. Doug Avatar, CapAm. Ellen Desp, Frozen. Frank Avatar, CapAm, Ender. Gary Avatar, Ender. New movie Ghost in the Shell was watched by Alice, Doug, Frank, Gary. Use KNN, $k = 3$, Euclidean. Hint, use watch history as features.

Step 1. Decide what a sample and a feature are

We classify movies, so each movie is a sample. The only info is who watched it, so each user is a feature. Entry is 1 if that user watched the movie, else 0. Order A B C D E F G.

Step 2. Write every movie as a vector, and its distance to Ghost

Ghost (?)	1001011	
Avatar (sci)	0001011	$\sqrt{1} = 1$
Big Hero 6 (ani)	1110000	$\sqrt{5} = 2.24$
Captain Am. (sci)	1001010	$\sqrt{1} = 1$
Despicable (ani)	1110100	$\sqrt{6} = 2.45$
Ender's Game (sci)	0010011	$\sqrt{3} = 1.73$
Frozen (ani)	0100100	$\sqrt{6} = 2.45$

Step 3. Why the distances are quick

$d = \sqrt{\sum (x_k - y_k)^2}$. With 0/1 entries each term is 1 where the two vectors disagree and 0 where they agree, so d^2 is just the count of users that differ. Avatar vs Ghost differ only at Alice, so $d = 1$.

Step 4. Take the 3 nearest and vote

Nearest are Avatar 1, Captain America 1, Ender's Game 1.73. All three are sci-fi, vote 3 to 0, so Ghost is **sci-fi**.

Problem 2. Multi-class evaluation

The question

3-class confusion matrix, rows are truth, columns are prediction.

	P1	P2	P3
T1	28	1	1
T2	3	25	2
T3	4	3	23

(a) accuracy with work, fractions OK. (b) per-class confusion matrices, then macro-averaged precision. (c) the micro confusion matrix, then micro-averaged precision.

(a) Accuracy

Correct predictions sit on the diagonal, 28+25+23 = 76. Total is every cell. Each row sums to 30, so 90 (add the rows twice). Acc = 76/90 = 38/45 ≈ .844.

(b) Step 1. Turn each class into a yes/no problem

Treat class c as positive and the other two as negative. TP is the diagonal cell. FN is the rest of row c (truly c, predicted other). FP is the rest of column c (predicted c, truly other). TN is the remainder, 90−TP−FN−FP.

	TP	FN	FP	TN
C1	28	1+1=2	3+4=7	53
C2	25	3+2=5	1+3=4	56
C3	23	4+3=7	1+2=3	57

As tables, C1 $\left[\begin{smallmatrix}28 & 2\\ 7 & 53\end{smallmatrix}\right]$, C2 $\left[\begin{smallmatrix}25 & 5\\ 4 & 56\end{smallmatrix}\right]$, C3 $\left[\begin{smallmatrix}23 & 7\\ 3 & 57\end{smallmatrix}\right]$ (rows truth +/−, columns pred +/−).

(b) Step 2. Precision per class, then average

Precision = $TP/(TP+FP)$, of everything predicted c, how much was right. 28/35 = .800, 25/29 = .862, 23/26 = .885. Macro is the plain mean, $\frac{1}{3}(.800 + .862 + .885) \approx .849$.

(c) Micro

Add the three tables cell by cell into one table, TP 76, FN 2 + 5 + 7 = 14, FP 7 + 4 + 3 = 14, TN 166, giving $\left[\begin{smallmatrix}76 & 14\\ 14 & 166\end{smallmatrix}\right]$. Micro precision = 76/(76 + 14) = 76/90 ≈ .844. It equals accuracy because every mistake is a FP for exactly one class.

Problem 3. Linear regression and overfitting

The question

Predict rent ($1000s) from size (1000 sq ft) and distance to campus (miles). Train apt1 size 1.2, dist 5, price 1.4. apt2 size 0.8, dist 3, price 2.1. Test apt3 size 2, dist 1, price 5. Model1 $[w_0, w_1, w_2] = [2.5, 1, -0.5]$, model2 $[1, 2, -0.3]$. (1) predictions for all 3 apartments by both models, (2) train RSS, (3) test RSS, (4) which model overfits.

Step 1. The model

$\hat{y} = w_0 + w_1 \cdot \text{size} + w_2 \cdot \text{dist}$. w_0 is the bias, it multiplies the constant 1.

Step 2. Plug in

Model1. apt1 $2.5 + 1(1.2) - .5(5) = 1.2$. apt2 $2.5 + .8 - 1.5 = 1.8$. apt3 $2.5 + 2 - .5 = 4.0$.
Model2. apt1 $1 + 2(1.2) - .3(5) = 1.9$. apt2 $1 + 1.6 - .9 = 1.7$. apt3 $1 + 4 - .3 = 4.7$.

Step 3. RSS = $\sum(y - \hat{y})^2$

Train (apt1, apt2). M1 $(1.4 - 1.2)^2 + (2.1 - 1.8)^2 = .04 + .09 = .13$. M2 $(-.5)^2 + (.4)^2 = .25 + .16 = .41$.
Test (apt3). M1 $(5 - 4)^2 = 1$. M2 $(5 - 4.7)^2 = .09$.

Step 4. Decide

Overfitting means low error on training data but high error on unseen data. M1 has the lower train RSS (.13 vs .41) and the much higher test RSS (1 vs .09), so **model1 overfits**.

Problem 4. PCA when rows are samples

The question

$X = \left[\begin{smallmatrix}1 & 4 & 7 & 10\\ 2 & 5 & 8 & 11\\ 3 & 6 & 9 & 12\end{smallmatrix}\right]$, rows samples, columns features (3 samples, 4 features). (1) Write the centered $\tilde{X}$. (2) Given U_A (3 × 3) from SVD of $\tilde{X}$ and U_B (4 × 4) from SVD of $\tilde{X}^\top$, with first column of $U_B = [-.1, -.3, -.5, -.7]^\top$, reduce to 1 dimension and explain.

Step 1. Center each feature

Features are columns, so subtract each column's mean. Means 2, 5, 8, 11. $\tilde{X} = \left[\begin{smallmatrix}-1 & -1 & -1 & -1\\ 0 & 0 & 0 & 0\\ 1 & 1 & 1 & 1\end{smallmatrix}\right]$.

Step 2. Get into Zhu's layout

Zhu's recipe wants features on rows, samples on columns ($p \times n$). Here that matrix is $\tilde{X}^\top$, which is 4 × 3.

Step 3. Pick the right U

G must turn 4 features into 1, so it is 4 × 1, which can only come from a 4 × 4 U. That is U_B (SVD of $\tilde{X}^\top$). U_A is 3 × 3, sample space, wrong.

Step 4. Keep $k = 1$ column

$G = U_k$ = first column of $U_B = [-.1, -.3, -.5, -.7]^\top$.

Step 5. Project, $Z = G^\top \tilde{X}^\top$

Sizes $(1 \times 4)(4 \times 3) = 1 \times 3$, one number per sample. Sample $1 = (-.1)(-1) + (-.3)(-1) + (-.5)(-1) + (-.7)(-1) = 1.6$. Sample 2 is all zeros so 0. Sample 3 $= -1.6$. Answer [1.6, 0, −1.6]. Use the given U, don't recompute.

Same question, flipped

PCA, rows are features (Zhu's layout)

$X = \left[\begin{smallmatrix}1 & 3 & 5\\ 2 & 2 & 2\end{smallmatrix}\right]$, rows are features, so 2 features and 3 samples. Given $u_1 = [1, 0]^\top$, reduce to 1D.

1. Layout check. Rows are features, which is already Zhu's $p \times n$, so no transpose.
2. Center each *row*. Means 3 and 2, $\tilde{X} = \left[\begin{smallmatrix}-2 & 0 & 2\\ 0 & 0 & 0\end{smallmatrix}\right]$.
3. U comes from SVD of $\tilde{X}$ and is 2 × 2, one row per feature. $G = u_1$ is 2 × 1.
4. $Z = G^\top \tilde{X}$, $(1 \times 2)(2 \times 3) = 1 \times 3$. Sample $1 = 1(-2) + 0(0) = -2$, then 0, then 2. Answer $[-2, 0, 2]$.

Confusion matrix with rows = prediction

If the table says rows are *predicted* and columns are *true*, e.g. $\left[\begin{smallmatrix}28 & 3 & 4\\ 1 & 25 & 3\\ 1 & 2 & 23\end{smallmatrix}\right]$, everything swaps.

1. Read the labels on the table before computing anything.
2. Precision$_c$ = diagonal / *row* sum (everything predicted c), 28/35, 25/29, 23/26.
3. Recall$_c$ = diagonal / *column* sum (everything truly c), 28/30, 25/30, 23/30.
4. Accuracy and micro scores don't change, diagonal / total = 76/90.

Weighted KNN with similarity instead of distance

$k = 3$, neighbor A with similarity .9, two B neighbors with .2 and .1.

1. Similarity goes up when closer, so use e^{+s}, not e^{-d}. $e^{.9} = 2.460$, $e^{.2} = 1.221$, $e^{.1} = 1.105$, sum 4.786.
2. Weights .514, .255, .231. A gets .514, B gets .486.
3. Predict **A**, even though the plain vote is 2 to 1 for B.

Cosine vs Euclidean, which neighbor is nearest

Test [1, 1]. P = [2, 2] label X, Q = [1, 0] label Y. 1-NN?

1. Euclidean is a distance, pick the *smallest*. P $\sqrt{2} = 1.41$, Q 1, so Q, label Y.
2. Cosine is a similarity, pick the *largest*. P $\frac{2+2}{\sqrt{2}\sqrt{8}} = 1$, Q $\frac{1}{\sqrt{2}\cdot 1} = .707$, so P, label X.
3. Same data, different answer, because cosine ignores length and only sees direction.

Small examples, question then answer

Weighted KNN. $k = 3$, one green neighbor at $d = 1$, reds at 3 and 5. Class?

$e^{-1} = .368$, $e^{-3} = .050$, $e^{-5} = .007$, sum .424. Weights .867, .117, .016. Green .867 beats red .133, so green, even though a plain vote says red.

Entropy split. Parent 3+, 3−. Split gives (3+,1−) and (0+,2−). Gain?

Parent $H = 1$. Left $H(\frac{3}{4}) = .811$, weight $\frac{4}{6}$. Right is pure, 0. Gain $= 1 - \frac{4}{6}(.811) = .459$.

One SGD step. $\mathbf{x} = [1, 2]$ (with bias 1), $y = 3$, $\mathbf{w} = [0, 0]$, $\alpha = .1$

$\mathbf{g} = (\mathbf{x}^\top \mathbf{w} - y)\mathbf{x} = (0 - 3)[1, 2] = [-3, -6]$. $\mathbf{w} \leftarrow \mathbf{w} - \alpha\mathbf{g} = [.3, .6]$.

Convolution. Vertical edge filter on a patch whose columns are 10, 10, 0

Multiply cell by cell and add. Left column $10(1+2+1) = 40$, middle ×0, right 0. Output 40. Output size of 7×7, 3×3 filter, stride 2, pad 1 is $\frac{7-3+2}{2} + 1 = 4$.

z-score and min-max of GPA 4, 3.5, 3

$\mu = 3.5$, $\sigma = \sqrt{(.25 + 0 + .25)/(3 - 1)} = .5$, z = $1, 0, -1$. Min-max = $1, .5, 0$.

Metrics. TP 6954, FN 46, FP 412, TN 2588

Acc 9542/10000 = .954. P 6954/7366 = .944. R 6954/7000 = .993. F1 $2PR/(P + R) = .968$.

OLS. Fit $y = w_0 + w_1 x$ to $(0, 1), (1, 3), (2, 5)$

X rows $[1, x]$. $X^\top X = \left[\begin{smallmatrix}3 & 3\\ 3 & 5\end{smallmatrix}\right]$, $X^\top \mathbf{y} = [9, 13]$, inverse $\frac{1}{6}\left[\begin{smallmatrix}5 & -3\\ -3 & 3\end{smallmatrix}\right]$, $\mathbf{w} = [1, 2]$.

Pseudocode

KNN(train, $\mathbf{x}$, K)
  for each n, $d_n \leftarrow \|\mathbf{x} - \mathbf{x}_n\|_2$
  $S \leftarrow$ the K smallest d_n
  for each class c, $v_c \leftarrow \sum_{k \in S} \mathbb{I}(y_k = c)$
  return $\arg\max_c v_c$

GD(J, α, ϵ)
  init $\mathbf{w}$, while $\|\nabla J(\mathbf{w})\|_2 > \epsilon$ do $\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla J(\mathbf{w})$

PCA(X $p \times n$, k)
  $\tilde{X} \leftarrow X -$ row means, $U \leftarrow \text{SVD}(\tilde{X})$, $G \leftarrow$ first k cols, $Z = G^\top \tilde{X}$

Cross-val split train in S folds, for each value average the S fold errors, pick best, refit on all train, test once.

Trap words

min vs arg min. validation vs test. sigmoid vs softmax. precision (FP) vs recall (FN). macro vs micro. small k overfit. KNN non-parametric. ordinal OK as numbers. Celsius interval, Kelvin ratio. entropy 0 is pure. "always" is usually F. α is convergence, λ is overfitting. G is features ×k.